

DRAIN SINKHORN: Safe Elimination for Batched Entropic Optimal Transport*

Xinyang Wen

Abstract

Fast OT backends reduce the cost of one Sinkhorn update, but a static batch still runs at full width until its slowest problem finishes. We present DRAIN SINKHORN, an execution layer for batched entropic optimal transport that physically removes completed problems and narrows subsequent updates. The candidate-axis kernel applies the original per-instance Sinkhorn map independently to every lane. After an alternating scaling cycle, one marginal is current by construction; a batched check of the other marginal filters the lanes that require the baseline two-sided stopping check. Passing lanes are removed from every candidate-indexed tensor, while unfinished lanes continue.

The mathematics characterizes removable work. For completion depths $\ell_1, \dots, \ell_W$, static execution spends $W\ell_{\max}$ candidate-update slots, whereas shrinking active execution spends exactly $\sum_t \ell_t$. A survival-curve formulation shows that positive removable work requires within-window disagreement in completion depth. A quotient Perron–Frobenius analysis gives a local mechanism for this disagreement: the finite-tolerance tail depends on the complete modal spectrum and proposal alignment, not on the slowest eigenvalue alone.

DrainSinkhorn delivers consistent backend-matched gains on real workloads. Under a matched 10^{-3} two-sided residual contract, its complete Flash-backed OT path is $4.110\times$ faster than public FlashSinkhorn serial on MetroPT-3 (95% CI [4.104, 4.114]) and $3.798\times$ faster on ImageNet-32 feature couplings. A two-host Packer19 sweep holds the current-residual controller fixed across five tolerances: the complete path is $1.250\text{--}1.270\times$ faster than public serial, and candidate-axis static packing is $1.258\text{--}1.280\times$ faster than width-one execution. When completion depths differ, physical compaction adds $1.480\text{--}1.638\times$ over logical masking; when every lane finishes together at 10^{-2} , that contrast is neutral. The same shrinking active-set principle improves matched OTT-JAX and PyKeOps paths. Numerical parity, achieved residuals, and application endpoints supply the implementation evidence; all reported consumer and training-quality gates pass.

1 Introduction

GPU implementations of entropic optimal transport (EOT) have become markedly better at executing a single Sinkhorn update. Matrix-free operators, tiled reductions, and fused streaming kernels reduce memory traffic and avoid materializing the transport matrix [4, 5, 14]. Many workloads, however, prepare a batch of couplings for predictive maintenance, scientific state transitions, or mini-batch training. Once the inner update is fast, total cost also depends on how long each problem remains in that batch.

Different problems usually reach the stopping tolerance at different iterations. A static batch follows the deepest solve: completed candidates continue to occupy state, audit bandwidth, and physical kernel lanes. A logical mask freezes their state but does not shrink the tensor seen by a fused kernel. The unused rectangle between each candidate’s completion depth and the deepest solve is therefore a distinct source of OT work.

We introduce DRAIN SINKHORN, an execution layer that removes this padding. Its name refers to the complete active-packing pipeline, which removes a problem after it passes the stopping check used by the selected solver. Candidate-axis packing applies the original per-instance update independently to each lane. A full alternating cycle makes the newly updated marginal current in exact arithmetic, so the other marginal provides a cheap batched screen. Lanes that pass the subsequent two-sided check are removed from potentials, marginals, support descriptors, and scratch buffers. The next backend update runs at the width of the unfinished set.

This design separates inner-kernel acceleration from cross-problem execution. FlashSinkhorn lowers the IO cost of one stabilized update [14]; DRAIN SINKHORN lowers the number and physical width of updates executed across a batch. We implement the shrinking active set over Flash, OTT-JAX, and PyKeOps. The full Flash path additionally exploits the updated-marginal screen; the OTT-JAX and PyKeOps paths realize the shrinking active set with independent backend mechanisms.

The mathematical object behind the system is the completion-depth profile. If lane t exits after ℓ_t packed updates, static execution consumes $W\ell_{\max}$ candidate-update slots and shrinking active execution consumes exactly $\sum_t \ell_t$. This task-independent identity depends only on the observed completion depths. A probabilis-

*Code: github.com/cis-aconiticacid/DrainSinkhorn

tic form sharpens the condition: the expected removable work is positive precisely when a window has positive probability of containing both completed and unfinished lanes at some depth. Hardware speedup then depends on the measured width-cost curve and the overhead of checking and compaction.

The design was motivated by parallel PageRank: positive fixed-point dynamics make completion structure a computational signal. LTGNN likewise uses a personalized-PageRank fixed-point formulation to scale recommendation training [16]. For Sinkhorn, quotient Perron–Frobenius geometry is central to the execution design because it gives a local account of unequal completion depths. The full modal response depends on both eigenvalues and proposal projections; a slow eigenvalue alone can badly overestimate a finite-tolerance tail. The solver’s stopping check supplies the completion observations, and the executor converts the measured depth profile into a shrinking physical batch.

FlashSinkhorn established the prior state of the art for online GPU EOT against GeomLoss tensorized, KeOps, and OTT-JAX backends [14]. We use its public serial path as the main reference and compose its fused Triton, IO-aware, linear-memory updates with physical active-set contraction. DrainSinkhorn extends that fast inner solver with active execution of heterogeneous EOT batches. OTT-JAX and PyKeOps provide independent backend-matched controls. The experiments distinguish OT-stage acceleration from application endpoints: the former measures the work removed inside EOT, while the latter tests whether that acceleration survives unchanged consumer or model work.

Our contributions are:

1. **An active-packing algorithm for batched Sinkhorn.** We combine candidate-axis packing of independent Sinkhorn maps with verifier-gated candidate retirement and physical compaction of all candidate-indexed state. Unlike static packing or logical masking, each subsequent kernel executes at the physical width of the unfinished set.
2. **A depth-to-work theory.** An exact survival-curve identity counts removed candidate-update slots, and a task-independent random-depth result identifies within-window completion disagreement as the condition for positive expected removable work. Quotient PF analysis supplies the local convergence basis for treating completion depth as the scheduling object.
3. **Causal component evidence.** Two-host Packer19 controls separate batched checking, Sinkhorn-specific screening, logical masking, and physical compaction. Fixed-work grouping and width interventions identify the heterogeneity and batch-width regimes in which physical active-set contraction pays.
4. **Backend and endpoint evidence.** MetroPT-3, Packer19, and ImageNet-32 evaluate the complete Flash path; OTT-JAX and PyKeOps test transfer on

ImageNet-32 and A2D2. The resulting paths achieve state-of-the-art execution performance on the tested heterogeneous batched-EOT workloads within each matched backend family. Numerical behavior is reported through parity, residual, and finite-precision stress tests, while complete consumers test application-visible output and training quality.

Fast kernels and active-set contraction attack different terms in runtime: the former makes an update cheaper, while the latter removes updates that belong only to already completed problems.

2 Setting and Prior Systems

Entropic optimal transport. For positive unit-mass marginals $a \in \mathbb{R}_+^n$ and $b \in \mathbb{R}_+^m$, EOT solves

$$\min_{\pi \geq 0} \langle C, \pi \rangle + \varepsilon \sum_{ij} \pi_{ij} (\log \pi_{ij} - 1), \quad \pi \mathbf{1} = a, \quad \pi^\top \mathbf{1} = b. \quad (1)$$

With $K = \exp(-C/\varepsilon)$, Sinkhorn alternates diagonal scalings to form $\pi = \text{diag}(u)K \text{diag}(v)$ [3]. Coordinate, relaxed, low-rank, and Newton methods reduce arithmetic or iteration count inside one solve [1, 7, 10, 11]. FlashSinkhorn rewrites stabilized updates as streaming LogSumExp reductions and lowers their IO cost [14]. DRAIN SINKHORN keeps the chosen inner update and changes the physical set of problems that receive the next one.

Initialization and repeated couplings. Carry, analytic extensions, and learned dual proposals can reduce a new problem’s starting defect [2, 12]. Large-coupling Flow Matching also uses soft c -transforms when supports change [17]. These mechanisms reshape the completion-depth distribution. DrainSinkhorn accommodates both regimes: the formal Packer19 attribution uses the same fixed proposal in every arm, and the ImageNet-32 training experiment uses independent cold-start couplings.

Dynamic batching and batched solvers. Automatic batching tracks the program point of each member in a data-dependent program [9]. Orca schedules autoregressive inference one iteration at a time [15]. Ginkgo fuses uniform independent linear systems and records per-item convergence [6]. jaxipm redesigns IPOPT around heterogeneous iteration fusion and replacement of completed problems [13]. We target scalable GPU EOT execution with online pairwise interactions instead of a materialized cost tensor for every candidate.

DrainSinkhorn addresses the numerical structure of EOT batches. Alternating scaling exposes a one-sided screen; the existing two-marginal stopping check is evaluated on candidate outputs; physical compaction must preserve the lane correspondence of potentials, marginals, supports, and scratch state. Generic masking leaves those

EOT-specific state transformations and the physical width unchanged. Table 1 compares the batched object, completion signal, and execution action of prior systems. Our OTT-JAX and PyKeOps variants transfer the shrinking active set, whereas the full Flash realization also uses the updated-marginal screen.

OT consumers. OTT provides a broad transport API [4]. Multisample Flow Matching consumes an OT coupling to select training pairs [8]; these couplings can be prepared independently of network optimization [17]. We evaluate the complete coupling-to-consumer boundary for industrial endpoints and carry the same execution path through a fixed-update feature-space OT-FM training pipeline.

3 DrainSinkhorn

Active packing versus static packing. DrainSinkhorn integrates candidate-axis packing, batched screening, verifier-gated candidate retirement, and physical compaction as one active-packing algorithm. Candidate-axis packing first executes W independent Sinkhorn maps in one packed kernel. Static packing retains physical width W until the deepest solve completes. If lane t exits after ℓ_t packed updates, active packing applies the configured two-sided verifier, removes passing candidates, and physically compacts all candidate-indexed state, so packed update k executes at survivor width

$$A_k = |\{t : \ell_t \geq k\}|.$$

Logical masking records completion at width W ; active packing executes subsequent kernels at survivor width A_k . Width-one/static/active controls decompose the complete method: width-one versus static isolates candidate-axis packing, while static versus active isolates verifier-gated retirement and physical compaction.

3.1 Batching the original Sinkhorn map

Let F_t denote one full Sinkhorn cycle for problem t , including that problem’s marginals, kernel operator, and numerical representation. Candidate packing adds a program axis but no cross-candidate reduction. In ideal arithmetic, its defining algebraic property is

$$\tilde{F}(\text{pack}(z_1, \dots, z_W)) = \text{pack}(F_1(z_1), \dots, F_W(z_W)). \quad (2)$$

Thus the packed kernel preserves the EOT objective and per-instance Sinkhorn map while changing their execution layout. Verifier-gated retirement filters the candidate axis after a stopping check and applies Equation (2) to the survivors.

For one candidate, a Sinkhorn cycle performs

$$u^{k+1} = a \oslash (Kv^k), \quad v^{k+1} = b \oslash (K^\top u^{k+1}), \quad (3)$$

where $\oslash$ denotes elementwise division. Immediately after the second leg, exact arithmetic gives

$$v^{k+1} \oslash (K^\top u^{k+1}) = b. \quad (4)$$

The column marginal is current by construction; the row marginal $u^{k+1} \oslash (Kv^{k+1})$ can remain outside tolerance. DrainSinkhorn evaluates that row residual for all active lanes in one batched operation and sends only screen-positive lanes to the baseline two-sided stopping check.

3.2 Screen, verify, and compact

Let $\hat{r}_{t,k}^{\text{row}}$ be the batched screen value. A lane is screen-positive when $\hat{r}_{t,k}^{\text{row}} \leq \tau_{\text{screen}}$; otherwise it continues without materializing the more expensive plan-level check. For a screen-positive lane, the implementation recomputes both marginals and applies the configured stopping rule

$$r_{t,k} = \max\{\|\pi_{t,k}\mathbf{1} - a_t\|_1, \|\pi_{t,k}^\top \mathbf{1} - b_t\|_1\} \leq \tau_{\text{stop}}. \quad (5)$$

The C80 Flash campaign uses 9.5×10^{-4} for both the screen and this two-sided check and reports an output tolerance of 10^{-3} . A rejected lane remains active for a later audit. A false-positive screen costs an extra full check; a false-negative screen delays removal. These effects are measured by the shadow-replay experiment in section 5.

For each passing lane, DrainSinkhorn writes the output and compacts every candidate-indexed object: source and target descriptors when candidate specific, marginals, dual potentials, centered shifts, log weights, and scratch buffers. The next packed update sees only surviving lanes. The logical-mask control uses the same update, screen, two-sided check, tolerance, and observed completion times. It freezes a passing lane’s potentials but retains the lane in the physical candidate tensor, so subsequent kernels still execute at the original width. Logical masking versus physical compaction therefore isolates the work removed by shrinking the physical active set.

Equation (2) and Equation (4) define the ideal-arithmetic transformation. Fixed-work parity, achieved residuals, near-threshold stress tests, and application endpoints characterize its implementation behavior.

Verifier-gated semantic safety. We use *safe* in one operational sense: a candidate is removed only after passing the same configured two-sided stopping check used by the selected backend. The one-sided screen decides only whether to invoke that check; it cannot authorize removal. DrainSinkhorn therefore preserves the backend’s configured release authority while changing how unfinished candidates are scheduled and stored. The timed Flash path applies this rule in the precision supported by that backend. FP64 appears only as an independent replay and audit reference in the separate C10 numerical stress test; it is not part of the timed Flash execution path.

Table 1: DrainSinkhorn specializes active execution to Sinkhorn state. The table records each system’s completion signal and physical action rather than treating a shared batch dimension as the contribution.

System	Batched numerical object	Completion signal	Execution action	Output check
Ginkgo CG [6]	Uniform independent linear systems	Per-system convergence state	Skips work for a converged system inside the fused solver	Linear-system residual criterion
jaxipm [13]	Independent nonlinear programs	Heterogeneous iteration fusion	Replaces completed problems to sustain throughput	Nonlinear-program convergence checks
DRAINSinkhorn	Positive EOT problems with compatible packed state	One-sided Sinkhorn screen followed by the configured two-sided check	Removes passing lanes from potentials, supports, marginals, and scratch buffers	Current two-marginal residual tolerance

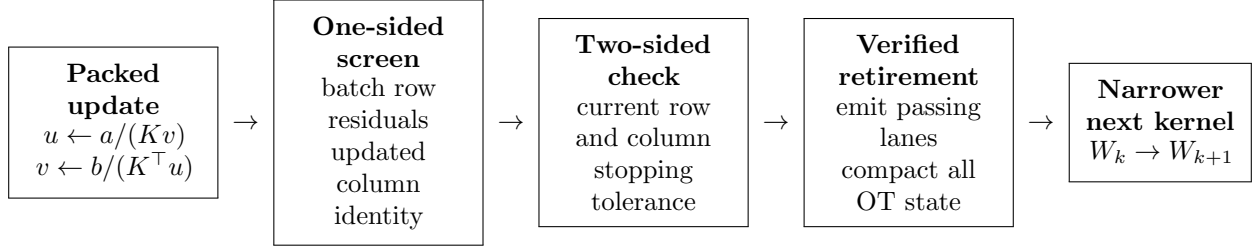


Figure 1: DrainSinkhorn’s execution loop. The one-sided screen avoids many full plan checks. Passing lanes leave every candidate-indexed tensor; the next packed update processes only the survivors.

3.3 Execution modes and fallback

The implementation exposes four execution modes. Width-one executes the project backend one problem at a time. Fixed-width packing updates every lane until a common final boundary. Logical masking records per-lane completion and freezes completed states without narrowing the kernel. Physical compaction removes passing lanes. Width determines whether a packed kernel is efficient, and the completion-depth survival curve determines how much width physical compaction can remove. Formal campaigns fix execution mode, width, audit interval, and fallback before timing. C35 and M3 measure the operating surface. Numerical anomalies and iteration-limit events return to the matched sequential path.

When a complete problem fits on one accelerator, different windows are sent to independent complete-device workers, giving each device an independent update path and avoiding per-update collectives. Row sharding remains a capacity path for a single oversized problem; the main scale-out path assigns complete problems to independent devices.

4 Completion Depths: Convergence and Eliminated Work

The executor is organized around completion depth rather than a single worst-case iteration count. The identities below convert observed completion depths into removable work. The nonlinear Perron–Frobenius analysis is central to this design: it shows locally why finite-tolerance depths depend jointly on contraction modes and the proposal’s projection onto those modes, motivating an executor that follows the measured survivor set instead of assuming uniform depth. Release authority remains the configured

two-sided stopping check.

4.1 Exact depth-to-work identities

Let ℓ_t be the first production audit at which candidate t passes the configured two-sided check, and let

$$A_k = |\{t : \ell_t \geq k\}| \quad (6)$$

be the surviving width before packed update k . The measured stopping check supplies both quantities.

Proposition 1 (Depth-profile work identity). *For a finite window of W candidates,*

$$S_{\text{active}} = \sum_{k=1}^{\ell_{\max}} A_k = \sum_{t=1}^W \ell_t, \quad S_{\text{static}} = W \ell_{\max}. \quad (7)$$

Hence physical active-set contraction removes exactly

$$R = W \ell_{\max} - \sum_{t=1}^W \ell_t \quad (8)$$

candidate-update slots relative to a fixed-width batch with the same final boundary.

Proof. Use $\ell_t = \sum_{k \geq 1} \mathbf{1}\{\ell_t \geq k\}$ and exchange the two finite sums. Fixed-width execution keeps all W lanes through $\ell_{\max}$. $\square$

Logical masking changes which states update but not the physical width seen by the kernel, so it retains $W \ell_{\max}$ slots. The area between the static rectangle and the survival curve is therefore the exact post-run work removed by compaction. Audit spacing is already reflected in the observed ℓ_t ; an ideal first-passage depth on a denser grid would define a different quantity.

Proposition 2 (Task-independent opportunity condition). *Let $(\ell_1, \dots, \ell_W)$ be any random vector of finite completion depths. Then $\mathbb{E}[R] > 0$ if and only if there exists a depth k such that*

$$\Pr(0 < A_k < W) > 0. \quad (9)$$

Proof. R is nonnegative and is zero exactly when every lane has the same completion depth. Unequal finite depths create a level k with both survivors and completed lanes, and the converse follows directly from the definition of A_k . $\square$

The condition depends on the joint depth law inside a packed window. Perfectly synchronized lanes create no removable work even when completion depths vary across windows.

4.2 Nonlinear Perron–Frobenius basis for local convergence

Sinkhorn scalings are not unique: multiplying one scaling by a positive constant and dividing the other by the same constant leaves the coupling unchanged. In log coordinates this is the one-dimensional gauge direction $(f, g) \mapsto (f + c\mathbf{1}, g - c\mathbf{1})$. Its eigenvalue is one, but it does not represent a change in the transport plan. We therefore remove this direction and analyze the Jacobian on the gauge quotient; the nontrivial modes below start at $j = 2$.

At a strictly positive fixed point with coupling π^* , define the normalized transfer $P = D_a^{-1/2} \pi^* D_b^{-1/2}$. After the marginal-weighted change of coordinates, one linearized half-step acts by P between the two tangent spaces. The reverse half-step is its adjoint P^* . One full alternating Sinkhorn cycle composes the forward and reverse transfers, which gives the weighted quotient Jacobian $J^* = P^*P$. Hence J^* is positive semidefinite in the associated weighted inner product and admits modes $J^* \phi_j = \lambda_j \phi_j$.

Write the proposal error as $e_0 = \sum_{j \geq 2} \alpha_j \phi_j$, where $\alpha_j = \langle e_0, \phi_j \rangle_*$. Thus α_j measures proposal alignment: it is the amount of the initial error placed in mode j . In the tangent model, the squared fixed-point defect after k cycles is

$$\|(I - J^*)(J^*)^k e_0\|_*^2 = \sum_{j \geq 2} (1 - \lambda_j)^2 \alpha_j^2 \lambda_j^{2k}. \quad (10)$$

This expression also shows why λ_2 alone cannot predict a finite-tolerance completion depth. Its slow mode matters only when α_2 is appreciable; another mode can dominate at the relevant finite k , and the factor $(1 - \lambda_j)$ changes how eigenmodes appear in the measured defect. Outside the tangent regime, a nonlinear remainder and the conversion from quotient error to the actual marginal residual also affect the first tolerance crossing. The full spectrum and the proposal projections therefore provide a local

mechanism for depth variation, while the configured two-sided stopping check supplies the execution depths used by the algorithm.

4.3 Hardware realization and break-even

Wall time maps these slots through the backend’s width-dependent cost. Let $c_k(A_k, \xi_k)$ be the measured cost of update k , where ξ_k records support shape, data type, and backend state. Then

$$T_{\text{active}} = \sum_k c_k(A_k, \xi_k) + H_{\text{screen}} + H_{\text{check}} + H_{\text{compact}}, \quad (11)$$

while fixed-width execution replaces A_k with W and omits compaction. The break-even condition is

$$\sum_k [c_k(W, \xi_k) - c_k(A_k, \xi_k)] > H_{\text{screen}} + H_{\text{check}} + H_{\text{compact}}. \quad (12)$$

Equation (7) is exact work accounting. Measuring the conditional width-cost curve turns Equation (11) into a workload-specific timing model. Our experiments estimate this operating surface through direct interventions on width and grouping.

5 Experiments

All speedups are baseline time divided by DrainSinkhorn time. OT-stage time is measured directly or reconstructed by subtracting the paired consumer time from the encompassing recorded interval. For reconstructed rows, consumer time is subtracted within each paired timing unit before ratios, intervals, or bootstrap summaries are formed. Matched component comparisons hold inputs, proposal, tolerance, audit grid, two-sided check, and consumer fixed. Complete-path comparisons measure the deployed OT path on the same inner-backend family. When stopping controllers produce different achieved residuals, the achieved residuals and consumer-output deviations are reported together.

Accordingly, the reported ratios have three distinct roles:

$$\underbrace{\text{complete OT speedup,}}_{\text{complete-method value}} \underbrace{\text{width-one} \rightarrow \text{static,}}_{\text{packing attribution}} \underbrace{\text{static/logical} \rightarrow \text{active}}_{\text{retirement-compaction attribution}}.$$

The first ratio measures complete-path performance against an external reference; the latter two decompose DrainSinkhorn’s packed execution.

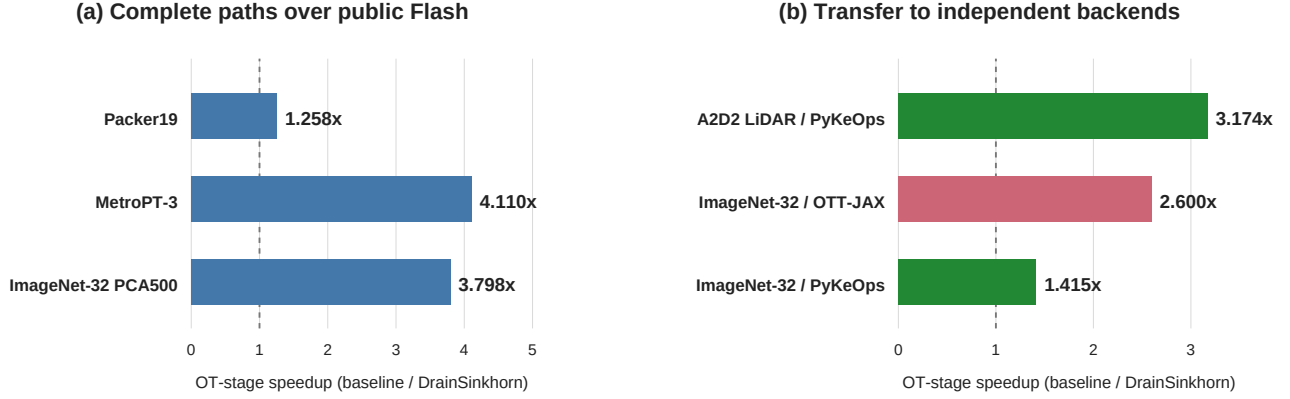


Figure 2: Complete-algorithm OT-stage acceleration on real workloads. Panel (a) compares the complete DrainSinkhorn path with the public Flash path. MetroPT-3 and ImageNet-32 use the same 10^{-3} stopping contract in both paths; Packer19 uses the same current two-sided residual controller at $\tau = 10^{-3}$. Panel (b) implements the execution principle independently over OTT-JAX and PyKeOps. Every bar is a complete OT-stage ratio; matched component effects are reported later in table 6.

Table 2: Complete OT-stage comparisons on real data across inner backends. Every ratio is baseline time divided by DrainSinkhorn time, and every uncertainty label states whether it is a bootstrap interval or an observed range. Timing repeats and GPU placements stabilize timing; they are not independent workload samples. For Packer19, H-O-G denotes host-order-GPU. The 24 Packer19 pairs share one frozen biological workload and quantify timing reproducibility, not biological-sample variability. Ratios below one mean that the external baseline is faster.

Backend	Workload	OT-stage contrast	Speedup (uncertainty)	Timing; unit	Accuracy contract
Flash	Packer19, $n = 16384, W = 8$	public same-controller / complete	1.258 $\times$ (95% CI [1.257, 1.259])	direct E1; 24 H-O-G pairs	identical 10^{-3} current-residual controller
Flash	MetroPT-3, $n = 16384, W = 16$	public serial / complete	4.110 $\times$ (95% CI [4.104, 4.114])	E2-consumer; 6 host-plan pairs	both residuals $< 10^{-3}$
Flash	ImageNet-32, $n = 1024, W = 8$	public serial / complete	3.798 $\times$ (seed range 3.704-3.983 $\times$)	E3 solver field; 3 seeds	identical 10^{-3} verifier
OTT-JAX	ImageNet-32, $n = 4096, W = 16$	native vmap / phase-active	2.600 $\times$ (repeat range 2.338-2.664 $\times$)	direct E1; 6 timing repeats	max L1 9.97×10^{-4}
PyKeOps	A2D2 LiDAR, $n = 8192, W = 8$	sequential carry / active	3.174 $\times$ (clip range 2.180-3.944 $\times$)	direct E1; 3 real clips	max L1 9.97×10^{-4}
PyKeOps	ImageNet-32, $n = 16384, W = 4$	sequential / active	1.415 $\times$ (seed range 1.360-1.448 $\times$)	direct E1; 3 seeds	max L1 8.93×10^{-4}

5.1 Complete-algorithm OT-stage acceleration

The primary matched-backend comparison uses public FlashSinkhorn serial on MetroPT-3 and ImageNet-32 feature couplings under the same EOT objective and 10^{-3} two-sided residual tolerance. FlashSinkhorn is the prior state-of-the-art online GPU EOT backend, with fused, IO-aware updates that outperform GeomLoss tensorized, KeOps, and OTT-JAX implementations [14]. DrainSinkhorn is 4.110 $\times$ and 3.798 $\times$ faster on those two matched cells. Packer19 supplies a controller-matched tolerance sweep; OTT-JAX and PyKeOps test backend transfer. These results establish execution gains within each matched backend.

Matched-controller complete paths. On MetroPT-3, the complete DrainSinkhorn OT path is 4.110 $\times$ faster than public Flash serial, with both paths ending in the 10^{-3} residual band. A reanalysis of the six valid host-plan units gives 95% CI [4.104, 4.114] with 6/6 wins. The matched static/active control assigns 1.754 $\times$ to verifier-gated retirement and physical compaction; candidate slots fall from 5696 to 3128, and OT-window energy falls by 1.783 $\times$.

In C63, every training step uses a real ImageNet-32

PCA500 feature coupling. Across three paired seeds, the complete DrainSinkhorn solver is 3.798 $\times$ faster than public Flash under the same 10^{-3} stopping tolerance; table 3 reports the four solver-phase times. The registered 2.34729 $\times$ width-one/static and 1.03556 $\times$ static/active ratios (range 1.02982-1.04311 $\times$) are measured on the complete fixed-update training interval, not on the solver field alone. They quantify how the two nested parts of DrainSinkhorn affect the C63 training endpoint: static packing is the fixed-width base, and verifier-gated candidate removal plus physical compaction turn that base into shrinking active packing. They do not partition the separate 3.798 $\times$ solver-only ratio.

The numerical quality results make the C63 endpoint check explicit rather than reducing it to a pass count. At NFE 4, 8, and 16, the complete method passes all nine predeclared non-inferiority comparisons against public Flash. The evaluation is a real ImageNet-32 PCA500 feature-space OT-FM endpoint; it does not claim pixel-space FID.

On Packer19, the complete DrainSinkhorn path is 1.258 $\times$ faster than public serial at $\tau = 10^{-3}$ (95% CI [1.257, 1.259]); all 24 paired units favor the complete path. Across the five registered residual tolerances, the corresponding ratio stays between 1.250-1.270 $\times$. This

Table 3: Absolute OT-stage time (seconds) for the three headline Flash workloads at the 10^{-3} operating point. Packer19 reports the median over 24 paired host-order-GPU units. ImageNet-32 reports solver-phase mean $\pm$ sample standard deviation over three paired training seeds. MetroPT-3 reports the median of the three registered worker-1 plan units, the common stratum in which public, static, and complete active paths retain absolute E1 fields; the headline MetroPT ratio instead uses the six valid units from two physical hosts. A dash means that the path was not run in the registered campaign; it does not denote zero time or a failed run. The final column is the complete DrainSinkhorn path, which includes verifier-gated candidate removal and physical compaction.

Workload	Public serial	Project width-one	Static packing	Logical masking	Complete DrainSinkhorn
Packer19 ($n = 16384, W = 8$)	1.632	1.639	1.296	2.038	1.299
MetroPT-3 ($n = 16384, W = 16$)	34.772	—	14.839	—	8.462
ImageNet-32 PCA500 training	440.33 ± 15.58	375.84 ± 6.94	122.17 ± 2.91	—	115.89 ± 0.83

Table 4: ImageNet-32 PCA500 feature-space quality after 50k training updates. Entries are three-seed means for public Flash serial (Public) and the complete DrainSinkhorn path (Complete). Lower is better for all three metrics. The predeclared non-inferiority tests use upper bounds of 1.02 for curvature, 1.01 for held-out strict-coupling FM MSE, and 1.10 for projected Fréchet; all nine Complete/Public tests pass. These are feature-space metrics, not pixel-space FID.

NFE	Curvature		Held-out FM MSE		Projected Fréchet	
	Public	Complete	Public	Complete	Public	Complete
4	5.4630	5.4448	1.07842	1.07859	1.3278	1.2886
8	2.8300	2.8356	1.07842	1.07859	1.5937	1.5224
16	1.4476	1.4528	1.07842	1.07859	1.9004	1.8136

comparison fixes the current two-sided residual controller, audit interval, input, and E1 endpoint. Consumer TV accompanies the solver ratio as an output-deviation diagnostic.

An independently frozen same-controller replication corroborates the $\tau = 10^{-3}$ result at $1.263\times$ (95% CI [1.262, 1.264]) with 24/24 paired wins.

Transfer across backend families. The non-Flash rows test the execution principle independently of the Flash kernel. Phase-active execution is $2.600\times$ faster than native OTT-JAX on real ImageNet-32 at $W = 16$; its matched static/active control assigns $1.545\times$ to phase-boundary retirement and rebucketing. On real A2D2 LiDAR, the PyKeOps active path is $3.174\times$ faster than sequential carry at $W = 8$. A real ImageNet-32 PyKeOps cell remains positive at $1.415\times$. The tiled-log implementation reaches $n = 65536$ without materializing candidate kernels and supplies the measured capacity boundary. Together, these paths establish state-of-the-art execution results on the tested heterogeneous batched-EOT tasks within their matched backend families.

5.2 Packer19 stopping and execution Pareto

The two-host EXP-PACKER19 campaign evaluates residual tolerances 10^{-4} , 3×10^{-4} , 10^{-3} , 3×10^{-3} , and 10^{-2} without aggregating across tolerances. All arms use the same current two-sided residual controller and the same tight-reference consumer output.

Static packing improves over width-one execution by

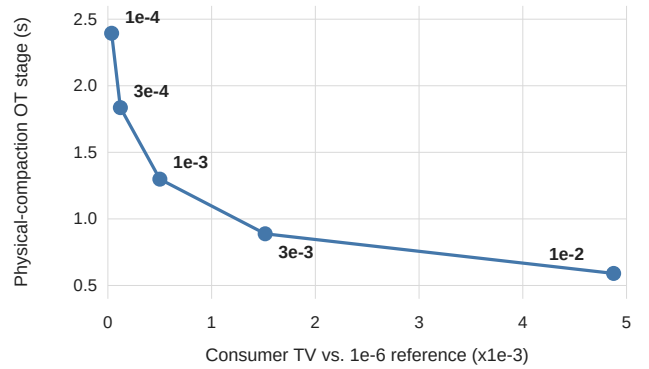


Figure 3: Tolerance-dependent Packer19 OT speed-output frontier for the physical-compaction path. Each point is the median E1 OT-stage time over 24 paired units at that residual tolerance; consumer TV is measured against the campaign’s 10^{-6} tight reference. The reference timing is excluded.

$1.258\text{--}1.280\times$ across all five tolerances, and the complete path improves over public same-controller serial by $1.250\text{--}1.270\times$. Physical compaction improves over logical masking by $1.480\text{--}1.638\times$ at 10^{-4} through 3×10^{-3} . At 10^{-2} , all eight lanes first pass at update 8, so there is no removable depth padding and LM/PC is neutral at $0.9995\times$ (95% CI [0.9987, 1.0003]). This null cell locates the measured boundary: uniform release depths leave no padding to remove.

The appendix maps the public native potential-change threshold to actual residual, consumer TV, and E1 time.

Table 5: Two-host Packer19 stopping-execution Pareto under a common current two-sided residual controller. Ratios are baseline time divided by candidate time. Each width-one/static and official/physical-compaction cell has 24/24 paired wins; LM/PC reports its equal-host bootstrap 95% interval. The 10^{-2} LM/PC cell is neutral because all lanes first pass together at update 8, leaving no completion-depth heterogeneity to exploit. Consumer TV is measured against the campaign’s tight reference and is an output-deviation diagnostic, not a biological-accuracy criterion.

τ	Width-one / static	Official / PC	LM / PC (95% CI)	Max actual residual	Max consumer TV
10^{-4}	$1.272\times$	$1.270\times$	$1.558\times$ [1.557, 1.559]	9.67×10^{-5}	3.61×10^{-5}
3×10^{-4}	$1.266\times$	$1.262\times$	$1.638\times$ [1.636, 1.639]	2.93×10^{-4}	1.20×10^{-4}
10^{-3}	$1.263\times$	$1.258\times$	$1.571\times$ [1.570, 1.573]	9.67×10^{-4}	5.01×10^{-4}
3×10^{-3}	$1.258\times$	$1.250\times$	$1.480\times$ [1.478, 1.482]	2.75×10^{-3}	1.52×10^{-3}
10^{-2}	$1.280\times$	$1.267\times$	$0.9995\times$ [0.9987, 1.0003]	9.48×10^{-3}	4.88×10^{-3}

Table 6: Packer19 single-variable attribution under one frozen input and release contract. Every row reports paired-consumer-subtracted OT-stage time, so the component effects share one measurement boundary.

Change	Matched contrast	Scope	Ratio	Evidence
Batched audit	serial / batched verifier	OT stage	$1.133\times$	24/24 wins
One-sided screen	two-sided / row screen	OT stage	$1.105\times$	24/24 wins
Logical freeze	fixed / logical mask	OT stage	$1.000\times$	95% CI [0.9998, 1.0003]
Physical compaction	logical mask / compaction	OT stage	$1.540\times$	95% CI 1.5389–1.5404

Batched-audit and row-screen rows have 24/24 paired wins; no separate E1 bootstrap interval is registered for those two contrasts.

Its table reports the mapping between two distinct stopping semantics. The complete application ratios for MetroPT-3 and ImageNet-32 feature OT-FM are $4.074\times$ and $2.786\times$; all endpoint quality gates pass. These E2/E3 ratios include unchanged consumer, model, or training work and are therefore lower than the corresponding OT-stage gains.

5.3 Matched component attribution

Packing and physical compaction. The earlier single-tolerance C80-LM control gives $1.540\times$ on Packer19, while the new five-tolerance campaign gives 1.480 – $1.638\times$ whenever completion depths differ. The matched MetroPT-3 control gives $1.754\times$. In the C63 ImageNet-32 four-arm training ablation, the complete fixed-update interval is $2.34729\times$ faster for static packing than project width one, while active packing with physical compaction adds $1.03556\times$ over static packing across three seeds (range 1.02982 – $1.04311\times$). These are E3 endpoint attributions; the C63 solver field is reported separately in table 3 and by the complete $3.798\times$ public/DrainSinkhorn ratio.

C80 and C80-LM use the same Packer19 single-cell transition workload on two physical hosts, each with four A100-SXM4-80GB GPUs. Three frozen method orders produce 24 paired host-order-GPU units. Each unit uses warmup-excluded repeat medians, the same input, Flash tree, 10^{-3} stopping tolerance, and transition consumer.

Checking and screening. The controls expose distinct work. Batched two-sided verification and the updated-marginal row screen reduce OT-stage time under matched

paths. Each C80 ratio subtracts its paired consumer time before forming the contrast.

Shadow replay evaluates the full two-sided check on all 1488 screen-negative events and finds 0 already-releasable lanes. The maximum difference between the screen and full-check row residual is 1.86×10^{-7} . Every emitted plan passes the two-sided 10^{-3} check. Shadow replay directly measures missed-ready events; the observed count is zero.

Logical masking versus physical compaction. In C80-LM, logical masking is neutral because the Flash kernel retains width eight. Physical compaction reduces the observed width, removes 100 of 256 physical candidate slots, and yields $1.540\times$ OT-stage acceleration with interval 1.5389 – $1.5404\times$. All 24 paired units favor compaction. Physical compaction also reduces median energy from 880 to 688 J, while its compacted buffers raise peak allocated memory from 80.8 to 106.9 MiB. EXP-PACKER19 extends this attribution over five tolerances and shows that the advantage disappears exactly when the release depths become uniform.

5.4 Operating regime: heterogeneity and physical width

M3 holds the set of 32 real MetroPT problems and total correction work fixed; only their grouping into windows changes. Oracle homogeneous grouping minimizes within-window depth variation. Heterogeneous and random groupings increase the area between the static rectangle and the release-depth survival curve. The matched static/active ratio grows from $1.105\times$ to $1.217\times$ and 1.235 – $1.271\times$ on random layouts.

C35 varies physical width on 40 paired real ImageNet-32 windows per cell. $W = 1$ is unprofitable, $W = 2$ is transitional, and every measured window wins at $W = 4, 8, 16$, reaching $1.621\times$ at $W = 16$. Together, M3 and C35 identify the two resources behind active-set contraction: heterogeneous release depths create removable work, and sufficient candidate width lets the backend exploit it.

Independent finite-precision stress test. C10 measures decision behavior on 84 near-threshold candidates

Table 7: M3 grouping intervention on one fixed real task multiset. For first-passage depths D_s within a window, $H_D = \text{sd}(D_s)/\text{mean}(D_s)$ and padding is $W \max_s D_s / \sum_s D_s - 1$. Active slots is the ratio of active to static candidate-iteration slots. The displayed values aggregate over windows; timing repeats stabilize each condition and are not independent datasets.

Grouping	H_D	Padding	Active slots	Static/active
heterogeneous	0.236	0.236	0.802	1.217×
homogeneous	0.127	0.110	0.892	1.105×
random 1	0.254	0.293	0.776	1.258×
random 2	0.264	0.270	0.789	1.235×
random 3	0.260	0.279	0.764	1.271×

Table 8: C35 real ImageNet-32 width sweep on the project Flash path. Each width contains 40 paired frozen windows; speedup is baseline/active with a paired 95% interval.

W	Speedup [95% CI]	Wins	Empirical regime
1	0.992 [0.980, 1.008]×	6/40	width one
2	1.073 [1.049, 1.097]×	33/40	transition
4	1.229 [1.195, 1.264]×	40/40	stable compaction
8	1.369 [1.327, 1.411]×	40/40	stable compaction
16	1.621 [1.487, 1.797]×	40/40	stable compaction

and 16 runtime attacks over 1/2/4/8 GPUs. Relative to independent FP64 replay, raw FP32 and TF32 each disagree on three accept decisions; an outward-bound guard followed by FP64 escalation has zero disagreements in these 100 cases. This guarded configuration is evaluated only inside C10. It is not the public Flash path and is not included in any timed application ratio.

Timed Flash applications and independent-worker scale-out. The timed Flash path uses the precision supported by the backend and the configured two-sided verifier; it does not invoke the C10 FP64 replay or escalation. All emitted plans meet the recorded two-sided residual, and every endpoint passes its registered output or quality gate. C67 assigns complete windows to independent GPU workers. Four workers achieve 3.965–3.979× fixed-per-worker throughput scaling on held-out MetroPT routes, with no Sinkhorn collective between workers.

6 Operating Regime and Composition

DRAIN SINKHORN applies to batched positive EOT problems that share a compatible packed representation and expose a two-sided residual. Its benefit is governed by the completion-depth survival curve and the backend’s response to physical width. The measured operating surface favors width-one execution for a single problem, static packing for nearly uniform depths, and physical compaction when depth disagreement exposes enough removable padding. C35, M3, and C80-LM measure these width and heterogeneity regimes. EXP-PACKER19 extends the

matched logical-mask/physical-compaction comparison across five residual tolerances: the gain is 1.480–1.638× before becoming neutral at 10^{-2} , where all lanes first pass on the same audit. Formal campaigns freeze the execution mode before timing. Irregular shapes are bucketed by compatible representation.

The active set composes with inner solvers that expose independent candidate state. One implementation reuses FlashSinkhorn’s shifted-potential formulation and Triton helpers and adds a candidate program axis. Independent OTT-JAX and PyKeOps implementations use phase re-bucketing and candidate-axis execution without the Flash kernel. Alternative initialization or inner acceleration can reduce a lane’s completion depth before the same physical shrinking step.

Fixed-work parity, residual replay, near-threshold stress tests, and consumer endpoints connect the ideal-arithmetic identities to production execution. The depth-to-work identity and hardware break-even model quantify the execution opportunity.

Physical compaction trades work for state movement and buffers. C80-LM measures about 26.1 MiB of additional peak allocation on Packer19 while wall time and energy fall. This cost enters deployment selection together with the observed survival profile and empirical width regime.

7 Conclusion

DRAIN SINKHORN removes padded OT work from batches with unequal completion depths. Candidate-axis packing evaluates the original per-instance Sinkhorn map; alternating scaling supplies a cheap one-sided screen; physical compaction narrows every subsequent candidate-indexed kernel. The survival-curve identity counts the candidate updates that disappear, while the joint-depth condition states when a random window contains positive expected removable work. The quotient modal response explains why finite-tolerance depths depend on proposal alignment as well as contraction rates.

Complete matched-controller OT-stage comparisons deliver 1.250–1.270× on Packer19 across five residual tolerances, 4.110× on MetroPT-3, and 3.798× on ImageNet-32 feature couplings. Independent OTT-JAX and PyKeOps implementations transfer the shrinking-active-set principle beyond Flash. These paths achieve state-of-the-art execution performance on the tested heterogeneous batched-EOT workloads within their matched backend families.

Matched controls identify the mechanism. Batched checking and the Sinkhorn-specific screen reduce inspection cost; logical masking leaves the physical kernel width unchanged; compaction removes OT work and energy. Across Packer19 tolerances it yields 1.480–1.638× over logical masking when completion depths differ and becomes neutral when all lanes finish together. Fixed-work grouping shows that greater within-window depth dis-

agreement creates more removable padding, and width sweeps show when the backend converts that work reduction into wall time. Across the reported applications, achieved residuals, consumer outputs, and training-quality checks are reported alongside speed. Fast inner kernels make each update cheaper; DrainSinkhorn stops issuing those updates to completed problems.

Reproducibility. The release artifact records source and input hashes, commands, software and GPU environments, raw outputs, numerical checks, paired analyses, and every public baseline used in the manuscript.

References

- [1] Jason Altschuler, Jonathan Niles-Weed, and Philippe Rigollet. Near-linear time approximation algorithms for optimal transport via Sinkhorn iteration. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://papers.nips.cc/paper_files/paper/2017/hash/491442df5f88c6aa018e86dac21d3606-Abstract.html.
- [2] Brandon Amos, Giulia Luise, Samuel Cohen, and Ievgen Redko. Meta optimal transport. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 791–813, 2023. URL <https://proceedings.mlr.press/v202/amos23a.html>.
- [3] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transportation distances. In *Advances in Neural Information Processing Systems*, volume 26, pages 2292–2300, 2013. URL <https://arxiv.org/abs/1306.0895>.
- [4] Marco Cuturi, Laetitia Meng-Papaxanthos, Yingtao Tian, Charlotte Bunne, Geoff Davis, and Olivier Teboul. Optimal transport tools (OTT): A JAX toolbox for all things wasserstein, 2022. URL <https://arxiv.org/abs/2201.12324>.
- [5] Jean Feydy, Thibault Séjourné, François-Xavier Vialard, Shun ichi Amari, Alain Trounev, and Gabriel Peyré. Interpolating between optimal transport and MMD using sinkhorn divergences. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pages 2681–2690, 2019. URL <https://proceedings.mlr.press/v89/feidy19a.html>.
- [6] Ginkgo Project. Batched solvers: Overview. Ginkgo user guide, 2026. URL <https://gko-project.org/user-guide/concepts/batched.html>. Accessed 2026-08-28.
- [7] Mete Kemert, Amir massoud Farahmand, and Allan D. Jepson. A truncated newton method for optimal transport, 2025. URL <https://arxiv.org/abs/2504.02067>.
- [8] Aram-Alexandre Pooladian, Heli Ben-Hamu, Carles Domingo-Enrich, Brandon Amos, Yaron Lipman, and Ricky T. Q. Chen. Multisample flow matching: Straightening flows with minibatch couplings. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 28100–28127, 2023. URL <https://proceedings.mlr.press/v202/pooladian23a.html>.
- [9] Alexey Radul, Brian Patton, Dougal Maclaurin, Matthew D. Hoffman, and Rif A. Saurous. Automatically batching control-intensive programs for modern accelerators. In *Proceedings of Machine Learning and Systems*, volume 2, 2020. URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/39945d578f616735572174bf5e8f155d-Abstract.html.
- [10] Meyer Scetbon, Marco Cuturi, and Gabriel Peyré. Low-rank Sinkhorn factorization. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9344–9354, 2021. URL <https://proceedings.mlr.press/v139/scetbon21a.html>.
- [11] Alexis Thibault, Lénaïc Chizat, Charles Dossal, and Nicolas Papadakis. Overrelaxed Sinkhorn-Knopp algorithm for regularized optimal transport, 2017. URL <https://arxiv.org/abs/1711.01851>.
- [12] James Thornton and Marco Cuturi. Rethinking initialization of the sinkhorn algorithm. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, pages 8682–8698, 2023. URL <https://proceedings.mlr.press/v206/thornton23a.html>.
- [13] John Viljoen, Johanna Haffner, Masayoshi Tomizuka, and Negar Mehr. Scaling nonlinear optimization: Many problems one GPU, 2026. URL <https://arxiv.org/abs/2606.26341>.
- [14] Felix X.-F. Ye, Xingjie Li, An Yu, Ming-Ching Chang, Linsong Chu, and Davis Wertheimer. FlashSinkhorn: IO-aware entropic optimal transport on GPU, 2026. URL <https://arxiv.org/abs/2602.03067>.
- [15] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, pages 521–538, 2022. URL <https://www.usenix.org/conference/osdi22/presentation/yu>.
- [16] Jiahao Zhang, Rui Xue, Wenqi Fan, Xin Xu, Qing Li, Jian Pei, and Xiaorui Liu. Linear-time graph neural networks for scalable recommendations. In *Proceedings of the ACM Web Conference 2024*, pages 3533–3544. ACM, 2024. doi: 10.1145/3589334.3645486. URL <https://doi.org/10.1145/3589334.3645486>.
- [17] Stephen Zhang, Alireza Mousavi-Hosseini, Michal Klein, and Marco Cuturi. On fitting flow models with large sinkhorn couplings, 2025. URL <https://arxiv.org/abs/2506.05526>.

A Measurement Contracts and Additional Detail

Measurement levels. E1 terminates at the measured coupling output or potentials after the configured two-sided stopping check. E2 includes the fixed downstream consumer. E3 includes training, generation, or task evaluation. C66 and C80 record E2 endpoints, and C63 records an E3 training endpoint. The main-text OT ratios extract E1 from those paired records: C66 and C80 subtract each unit’s paired consumer time before ratio formation and bootstrap, while C63 uses the recorded solver field. EXP-PACKER19 records E1 directly for every stopping-controller and execution-mode cell. Complete E2/E3 ratios are visualized in figure 4. The unchanged consumer or model time in E2/E3 dilutes the corresponding OT-stage gain. The C63 width-one/static and static/active ratios reported as internal training ablations are computed from the complete fixed-update E3 interval; they are not used to partition its separately recorded solver field.

C66 release-fair controls. Each workload runs independent Official public Flash serial, legacy static/active, and current static/active arms. Within the matched current contrast, static and active use the same inputs, cold proposal, regularization, tolerance, audit grid, two-sided check, packed backend, and fixed consumer; verifier-gated retirement and physical compaction are the controlled difference. Three order-balanced plan seeds are reused across four GPU placements. GPU placements and timing repeats stabilize end-to-end measurement; the registered endpoint ratio includes unchanged non-OT consumer time. The main-text C66 OT-stage ratio subtracts the paired consumer time within each host-plan unit. The plan seed remains the independent experimental unit; placements and repeats are timing replicas rather than independent MetroPT workloads. A reanalysis of the six valid physical-host-plan units reports a stratified bootstrap interval.

EXP-PACKER19 provenance. The stopping-execution protocol has SHA-256 c010bcf9b3e276c8269514324cc8ed4bc2fc5958c362c7b880e3946dee75075b and is bound to source commit 96de52b8ceff8fa3c0468bf45567e6b017483e73. Two physical hosts, four A100-SXM4-80GB GPUs per host, three frozen method orders, and three timing repeats produce 768 formal cells. Each tolerance has 24 paired host-order-GPU units and an equal-host bootstrap interval. Consumer TV is a numerical output-deviation diagnostic; the campaign establishes no biological tolerance requirement.

The table maps the public native potential-change controller to achieved marginal residual, consumer TV, and E1 time. Potential-change and marginal-residual thresholds have distinct stopping semantics. The formal causal ratios are those in table 5.

Table 9: Measured mapping from the public Flash native potential-change threshold to achieved two-sided residual, consumer-output deviation, and E1 OT time on Packer19. The native threshold is not a marginal-residual tolerance. For thresholds at most 3×10^{-4} , some lanes reach the registered iteration cap; those rows are plateau replay measurements rather than evidence that every lane triggered the native stopping rule.

Native threshold	Actual residual	Max consumer TV	E1 time (s)
10^{-2}	9.01×10^{-4}	8.83×10^{-4}	6.812
3×10^{-3}	1.93×10^{-4}	1.91×10^{-4}	11.403
10^{-3}	1.11×10^{-5}	2.08×10^{-6}	20.607
3×10^{-4}	1.18×10^{-6}	4.96×10^{-7}	40.625
10^{-4}	1.18×10^{-6}	4.96×10^{-7}	43.251
10^{-5}	1.18×10^{-6}	4.96×10^{-7}	46.285
10^{-6}	1.18×10^{-6}	4.96×10^{-7}	46.281

Table 10: Configuration of the main formal workloads. All use cold proposals and the current two-sided marginal release contract.

Campaign	Endpoint, numerical configuration, and statistical unit
C66	MetroPT-3 predictive maintenance (E2); $(n, W, \varepsilon) = (16,384, 16, .1)$; audit every 4; $\tau = 10^{-3}$; two hosts and three plans.
C80/LM	Packer19 scRNA transition (E2); $(16,384, 8, .1)$; audit every 4; $\tau = 10^{-3}$; 24 paired units.
EXP-P19	Packer19 same-controller Pareto (E1); $(16,384, 8, .1)$; audit every 4; $\tau = 10^{-4}$ – 10^{-2} ; 24 paired units per tolerance.
C63	ImageNet-32 PCA500 OT-FM (E3); $(1,024, 8, .03)$; audit every 4; $\tau = 10^{-3}$; three paired seeds.
C35	ImageNet-32 PCA500 width sweep (E1); $(4,096, 1-16, .1)$; audit every 5; $\tau = 10^{-3}$; 40 paired windows per width.

Execution-mode protocol. The formal campaigns fix representation, width, execution mode, audit interval, stopping tolerance, checking path, and fallback before timing. C35 and M3 map the width and depth-heterogeneity conditions for selecting static execution or physical active-set contraction.

Independent backend realizations. The OTT-JAX and PyKeOps paths implement active retirement through their native backend mechanisms. OTT-JAX executes fixed 20-update phases, checks at phase boundaries, retires passing lanes, and rebuckets survivors; native `vmap` and a matched fixed-phase static path provide its controls. The PyKeOps paths use one candidate-batched matrix-free operator, a fixed geometric audit schedule, two-sided residual stopping, and physical removal of passing candidate state. Sequential and static-padded PyKeOps paths use the same symbolic backend. The shared principle is per-candidate retirement from a shrinking active set; audit and packing mechanisms remain backend-specific. All cross-backend rows in table 2 report their achieved two-marginal residual.

Non-OT consumer/model time dilutes the OT-stage gain

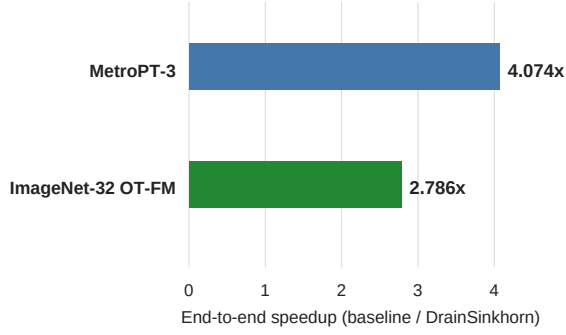


Figure 4: Complete application end-to-end speedups from the earlier endpoint campaigns. MetroPT-3 uses the same 10^{-3} residual contract, and all nine ImageNet-32 feature OT-FM checks at NFE 4/8/16 pass. These intervals include consumer, model, or training work outside OT.

Quotient PF mechanism. At a positive fixed point, the full-cycle Jacobian has the weighted-quotient form $J^* = P^*P$. For a tangent error $e_0 = \sum_{j \geq 2} \alpha_j \phi_j$, the modal defect is

$$\|(I - J^*)(J^*)^k e_0\|_*^2 = \sum_{j \geq 2} (1 - \lambda_j)^2 \alpha_j^2 \lambda_j^{2k}. \quad (13)$$

This expression explains why a λ_2 -only estimate can miss the observed finite-tolerance depth: the proposal may put little energy in the slowest mode, and modal dominance changes with k . Extending the tangent expression to a nonlinear trajectory requires a local remainder and an observable conversion; the paper uses it as a mechanism model and measures completion directly.

B Deployment and Operating Boundaries

Complete-device and row-sharded execution. When one problem fits on one accelerator, complete-device workers process different windows and require no Sinkhorn collective. Row sharding supplies a capacity path for an oversized single problem. Complete-device execution is the throughput path used by C67. On one host, increasing from one to four workers yields 3.965–3.979 $\times$ fixed-per-worker throughput scaling on two held-out MetroPT routes. The two-host, eight-worker fixed-per-worker run reports endpoint comparisons at that worker count. We report the one-to-four and two-host eight-worker results as distinct scaling studies. Peak memory remains local to each worker.